

pCGP: A Parallel Implementation of Cartesian Genetic Program-ming for Combinatorial Circuit Design and Time-Series Prediction

Abdul Manazir
Department of Computer Science,
Jamia Millia Islamia
New Delhi, India
abdulmanazir@gmail.com.

Khalid Raza
Department of Computer Science,
Jamia Millia Islamia
New Delhi, India
kraza@jmi.ac.in

Abstract—Cartesian Genetic Programming (CGP), a variant of genetic programming, is computationally extensive where its runtime is dominated by fitness evaluation, especially when the genotypes are complex. In order to speed up the execution, multiple offspring can be evaluated in parallel on multiple processing units. This paper presents a parallel implementation of CGP, called pCGP, using a multiple-threading scheme. The performance of pCGP is tested on two different standard benchmark datasets: i) classical digital circuit design problem, and ii) time-series dataset of monthly total sunspot number. The results show a significant reduction in the mean evolution time of parallel multi-threaded implementation over the serial single-threaded version of standard CGP. Hence, pCGP can be used for faster convergence in a complex problem having a large search space.

Keywords— *evolutionary algorithm, genetic programming, parallel computing, multi-threading, time-series forecasting*

I. INTRODUCTION

Cartesian Genetic Programming (CGP) is a variant of the Genetic Programming (GP) technique that utilizes Directed Acyclic Graph (DAG) for its representation (Miller and Thompson, 2000). The nodes are represented as two-dimensional grids, which is called ‘Cartesian’. Unlike GP, it is suitable to solve problems with multiple inputs and multiple outputs (MIMO) in nature. CGP also had an advantage over GP in that it does not exhibit the phenomenon of bloat. Generally, standard CGP uses mutation operators only (Miller, 2001). It has been reported that CGP outperforms GP (Koza, 1992). Programs in CGP are represented using computational nodes that form either one or two-dimensional structures and work as DAG. In CGP, a genotype is represented in the form of integers given by (Miller, 2011),

$$F_0C_{00}...C_{0a}F_1C_{10}...F_1C_{10}.....O_1O_2O_3....O_n$$

There are three types of gene nodes in CGP – function gene, connection genes, and output genes. Function gene ‘ F_i ’ provides the address of the function that the node represents in the function table, while connection genes ‘ C_{ij} ’ decides from where the input to the current node will be taken, and finally output genes ‘ O_i ’ define from which nodes outputs will be

taken. CGP also contains genes that neither affects the phenotype nor the fitness, called non-coding genes (Vassilev & Miller, 2000). A detailed and recent review on CGP can be found in Manazir & Raza (2019), Miller (2020), and Manazir & Raza (2022).

CGP, being an evolutionary algorithm, is computation-intensive, where its evolution is dominated by fitness evaluation, especially when the genotypes are complex (Gong et al., 2015). In order to speed up the computation, multiple offspring may be evaluated in parallel on multiple processing units as evaluation is independent of each other (Harter et al., 2017). The parallel CGP (pCGP) is usually based on master-slave model where a master node generates new individuals which are later evaluated by slave node. When pCGP waits until all the individuals in a generation are returned is called synchronous, on the other hand, asynchronous pCGP acts on each individual as it is returned (Harter et al., 2017). Following the (1+4) survival strategy as defined by Miller, returned individual is compared with the existing best in asynchronous pCGP. In case, a new individual is better than or equal to the current best, it becomes the current best. Following the strategy, new individuals are evolved using the current best via mutation, and the process continues until termination criteria are satisfied. Parallel implementation of GP has been extensive studies in the literature (Andre & Koza, 1996; Juillé & Pollack, 1996; Cano et al., 2011), however, parallel implementation of CGP is not well-studies and properly implemented.

There are several complex combinatorial problems having larger search space that needs parallel implementation of CGP. Some of the common problems addressed in the literature are (i) *n-bit parity problem* – a classical digital circuit problem, (ii) *evolutionary design of image filters*, (iii) *combinatorial adder and multiplier design*, and (iv) *various time-series prediction problems* (Miller, 2021). In this paper, we proposed a multithreaded (parallel) implementation of standard CGP, referred to as pCGP. The working of the pCGP is tested on both classical digital circuit design problems and the time-series benchmark dataset.

II. MATERIALS AND METHODS

In this paper, we propose thread-level parallelism for CGP in order to accelerate evolutionary steps such as genetic operator and fitness evaluation. The proposed parallel algorithm is shown in Algorithm 1 (Fig. 1). In this scheme, a new population is created using up to P number of threads, and the fitness evaluation step uses all these threads, and finally, new best individual is found by a single thread. Hence, parallelization is achieved in the first two steps. The algorithm begins with a randomly generated initial population and a while loop continue until a perfectly working circuit (in case of combinatorial circuit design problem) and root means square error (RMSE) below the acceptable range (in case of time-series prediction problem) is found. Over the evolutionary process, if a global best individual has a better fitness value than the local best individual, the global best value is seeded. The entire multi-threaded methodology has been implemented using Java Thread Library.

Algorithm 1. Parallel multithreaded implementation of CGP (pCGP)

```

Create random initial population;
while (termination_condition)
  for i = 1 to P
    if i = best_individual
      continue;
    end if
    individual[i] = individual[best_individual];
    mutate individual [i];
    evaluate individual [i];
  end for
  for i = 1 to P
    fitness[i] = 0;
  end for
  foreach test_vector
    for i = 1 to P
      if i = best_individual
        continue;
      end if
    end for
  end foreach
  for i = 1 to P
    individual [i].fit += fitness [i];
  end for
  for i = 1 to P
    if individual[i].fit > individual[best_individual].fit
      best_individual = i;
    end if
  end for
end while

```

Fig. 1. Algorithm for parallel multithreaded implementation of CGP (pCGP).

A. Datasets

In order to test the proposed parallel model, we considered two standard benchmark datasets: (i) classical digital circuit design problems such as combinational adders and multipliers, and (ii) time-series benchmark dataset from the Sunspot Index and Long-term Solar Observation (SILSO) of Solar Influences Data analysis Center (SIDC) (<https://wwwbis.sidc.be/silso/home>), containing monthly mean

total sunspot number from January 1900 to December 2020 (1440 time-points). We utilized 75% samples for training and the remaining 25% for testing.

III. RESULTS AND DISCUSSION

For the experimentation, we utilized the standard parameters: (1+4) evolutionary strategy, 10000 generations, node grids ranging from 7 x 7 to 10x10, and 50 independent runs for each training. The function set for the digital circuit problem was {nand, nor, and, or} and the function sets for the time-series forecasting problem was {+, -, *, /, sin, cos, exp, log}. The experiments were performed on Intel® i7-5500U comprises 8 cores CPU @ 2.40 GHz machine equipped with a Windows 10 operating system with 16 GB of RAM. The performance of the pCGP in terms of evolution time (mean, median, and standard deviation) is shown in Table 1 and Table 2.

For each input data, 50 independent runs were carried out, and mean, median, and standard deviation of evolution time were recorded for the combinatorial adder (Table 1, Fig. 2) and time-series forecasting problem (Table 2, Fig. 3).

TABLE I: PERFORMANCE OF COMBINATORIAL ADDER DESIGN PROBLEM

Width (bits)	Nodes ($n_r \times n_c$)	# of Threads	Time (s)		
			mean	median	standard deviation
7 x 7	1 x 150	1	1150.587	144.525	8.558
		8	120.556	109.050	9.556
8 x 8	1 x 200	1	1405.258	1380.447	25.325
		8	154.210	146.898	41.258
9 x 9	1 x 250	1	2814.254	2410.152	25.258
		8	542.258	505.245	45.257
10 x 10	1 x 300	1	3157.125	2854.245	87.254
		8	458.364	398.257	56.251

TABLE II: PERFORMANCE OF THE TIME-SERIES PREDICTION PROBLEM

Width (bits)	Nodes ($n_r \times n_c$)	# of Threads	Time (s)		
			mean	median	standard deviation
7 x 7	1 x 150	1	2558.551	2251.241	155.101
		8	810.255	795.105	250.125
8 x 8	1 x 200	1	2950.551	2810.550	235.254
		8	881.247	890.207	191.207
9 x 9	1 x 250	1	3154.251	3051.150	254.254
		8	914.235	890.132	345.185
10 x 10	1 x 300	1	3545.241	3315.201	220.125
		8	940.254	910.254	320.255

Table 1 presents the observation for the combinatorial adder problem, and Table 2 presents the observation for the time-series forecasting problem, with a width of 7-10 bits, computed over 50 independent runs, and tested on both single thread and 8 threads. For both the benchmark datasets, it has been observed that by increasing the number of threads, evolution time decreases, and convergence becomes more stable. Our results show the scalability of parallel implementation of CGP (pCGP) that can be further tested on a larger machine with hundreds of cores.

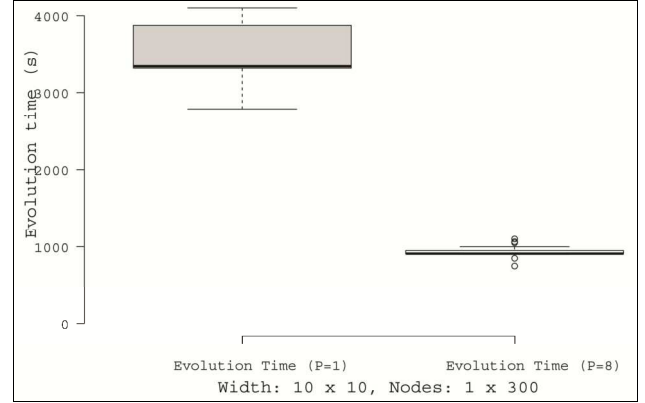
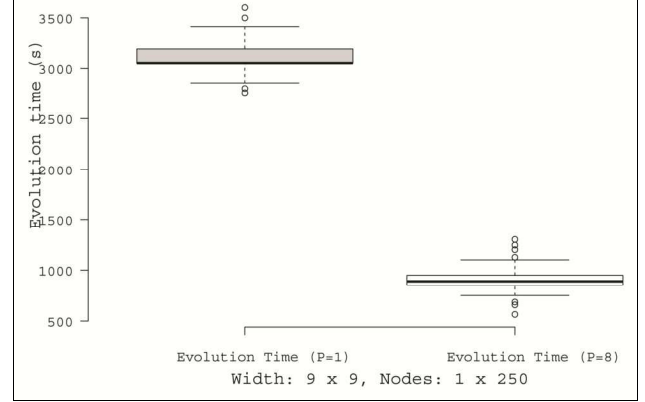
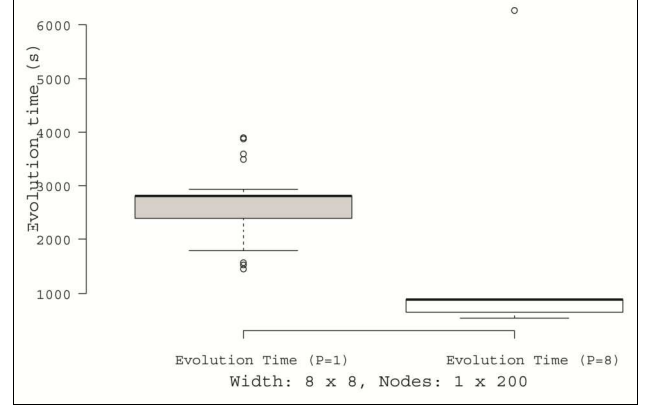
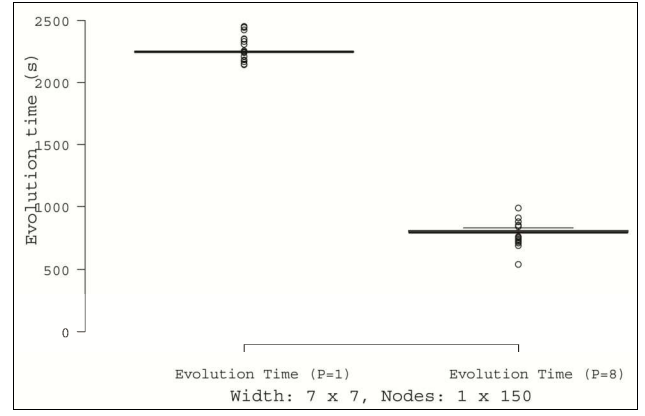
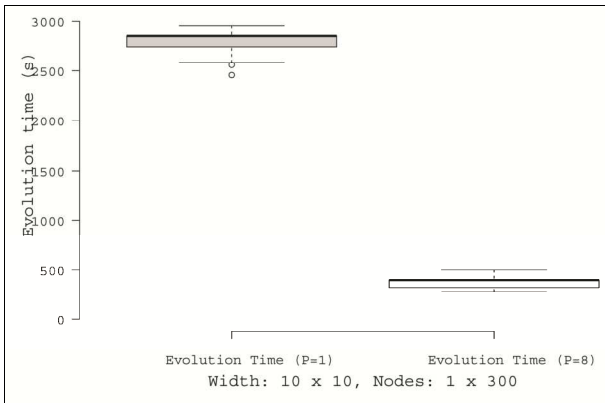
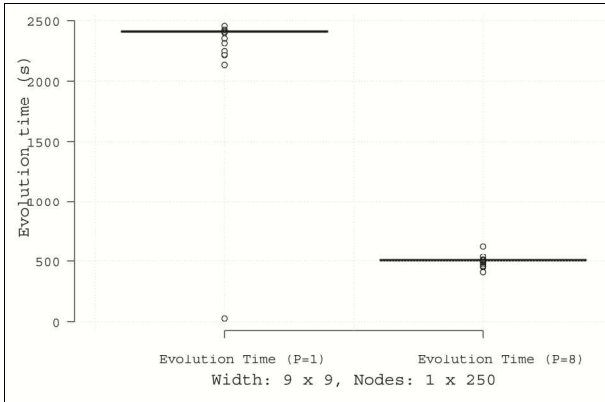
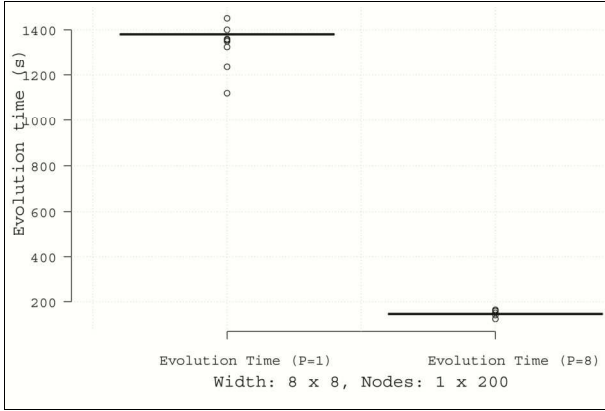
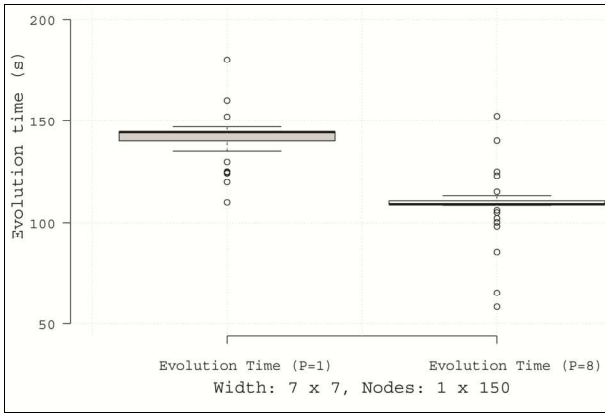


Fig. 2. Performance of pCGP on combinatorial circuit adder in terms of evolution time for single thread ($P=1$) and 8 threads ($P=8$) with different width, and nodes.

Fig. 3. Performance of pCGP on time-series sunspot forecasting dataset in terms of evolution time for single thread ($P=1$) and 8 threads ($P=8$) with different width, and nodes.

IV. CONCLUSION

Evolutionary algorithms are computation-intensive in nature due to several reasons including fitness evaluation. Cartesian Genetic Programming (CGP) is an evolutionary algorithm that has the potential to work as a multi-input and multi-output model. This paper presents a parallel implementation of CGP, called pCGP that speeds up the execution by evaluating multiple offspring in parallel on multiple processing units using a multiple-threading scheme. The test results show a significant reduction in the evolution time of the model while using a parallel implementation. Hence, pCGP can be used for faster convergence in a complex problem having a large search space. In future work, we can extend this work to multiple CPUs in a cloud-based environment.

LIST OF ABBREVIATIONS

CGP	: Cartesian Genetic Programming
DAG	: Directed Acyclic Graph
EA	: Evolutionary Algorithm
GP	: Genetic Programming
MIMO	: Multi-Input Multi-Output
pCGP	: Parallel CGP
RMSE	: Root Mean Square Error
SIDC	: Solar Influences Data analysis Center
SILSO	: Sunspot Index and Long-term Solar Observation

REFERENCES

- [1] Andre, D., & Koza, J. R. (1996). Parallel genetic programming: a scalable implementation using the transputer network architecture. In *Advances in genetic programming: volume 2* (pp. 317-337).
- [2] Cano, A., Zafra, A., & Ventura, S. (2011, May). A parallel genetic programming algorithm for classification. In *International Conference on Hybrid Artificial Intelligence Systems* (pp. 172-181). Springer, Berlin, Heidelberg.
- [3] Chitty, D. M. (2012). Fast parallel genetic programming: multi-core CPU versus many-core GPU. *Soft Computing*, 16(10), 1795-1814.
- [4] Gong, Y. J., Chen, W. N., Zhan, Z. H., Zhang, J., Li, Y., Zhang, Q., & Li, J. J. (2015). Distributed evolutionary algorithms and their models: A survey of the state-of-the-art. *Applied Soft Computing*, 34, 286-300.
- [5] Harter, A., Tauritz, D. R., & Siever, W. M. (2017, July). Asynchronous parallel cartesian genetic programming. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion* (pp. 1820-1824).
- [6] Juillé, H., & Pollack, J. B. (1996). Massively parallel genetic programming. In *Advances in genetic programming: volume 2* (pp. 339-357).
- [7] Koza, J.R. (1992). *Genetic Programming: On the Programming of Computers By Means of Natural Selection*, vol. 1. MIT Press, Cambridge.
- [8] Manazir, A., & Raza, K. (2019). Recent developments in cartesian genetic programming and its variants. *ACM Computing Surveys (CSUR)*, 51(6), 1-29.
- [9] Manazir, A., & Raza, K. (2022). Comparative Evaluation of Genetic Operators in Cartesian Genetic Programming. In *Proc. of 21st International Conference Intelligent Systems Design and Applications*, December 13-15, 2021. *Lecture Notes in Networks and Systems*, Springer, 418: 765-774.
- [10] Miller, J. (2001). What bloat? Cartesian genetic programming on Boolean problems. In *2001 Genetic and Evolutionary Computation Conference* (pp. 295-302).
- [11] Miller, J. F. (2000). Cartesian Genetic Programming: its status and future, *Genetic Programming and Evolvable Machines*, Vol. 21, pp. 129-168.
- [12] Miller, J. F. (2011). Cartesian genetic programming. In *Cartesian Genetic Programming* (pp. 17-34). Springer, Berlin, Heidelberg.
- [13] Miller, J. F. (2020, July). Evolving developmental neural networks to solve multiple problems. In *ALIFE 2020: The 2020 Conference on Artificial Life* (pp. 473-482). MIT Press.
- [14] Miller, J. F., & Thomson, P. (2000). Cartesian genetic programming. In *European Conference on Genetic Programming* (pp. 121-132). Springer Berlin Heidelberg.
- [15] Vassilev, V. K., & Miller, J. F. (2000). The advantages of landscape neutrality in digital circuit evolution. In *International Conference on Evolvable Systems* (pp. 252-263). Springer.